

ACTIVIDADES TEÓRICAS - FUNCIONES, STRING, VECTOR Y CLASES EN C++

octavio mazars 5*5ta

1. DIFERENCIA ENTRE FUNCIONES Y PROCEDIMIENTOS EN C++

En C++ no existe una distinción sintáctica formal entre "función" y "procedimiento" como en Pascal: todo se declara con la palabra clave que indica el tipo de retorno. Sin embargo, conceptualmente se sigue usando la distinción:

- **FUNCIÓN:** es un bloque de código que SÍ devuelve un valor mediante la sentencia "return". Su tipo de retorno es distinto de void.
 - Ejemplo: `int sumar(int a, int b) { return a + b; }`
- **PROCEDIMIENTO:** es un bloque de código que NO devuelve ningún valor. En C++ se implementa como una función cuyo tipo de retorno es void.
 - Ejemplo: `void mostrarMensaje() { cout << "Hola"; }`

CUÁNDO CONVIENE USAR CADA UNO:

- Se usa una **FUNCIÓN** cuando el objetivo del bloque es calcular o producir un resultado que después se necesita usar en otra parte del programa (por ejemplo, calcular un promedio, un área, validar un dato y devolver true/false).
- Se usa un **PROCEDIMIENTO** (función void) cuando el objetivo es ejecutar una acción o efecto (mostrar por pantalla, modificar un vector por referencia, cargar datos, etc.) sin que sea necesario devolver un valor directamente, aunque puede modificar parámetros pasados por referencia o puntero.

2. COMPONENTES DE UNA FUNCIÓN

Una función en C++ está compuesta por:

- **TIPO DE RETORNO:** indica el tipo de dato que devuelve la función (int, float, bool, void si no devuelve nada, etc.)
- **NOMBRE:** identificador único con el que se invoca la función.

- **PARÁMETROS (lista de argumentos):** variables entre paréntesis que reciben los valores con los que se invoca la función. Pueden pasarse por valor, por referencia (&) o como punteros (*).
- **CUERPO:** bloque de código entre llaves { } que contiene las instrucciones que se ejecutan.
- **SENTENCIA RETURN (si corresponde):** devuelve el valor calculado al punto donde se llamó la función. En funciones void es opcional y solo se usa para cortar la ejecución anticipadamente.

Ejemplo señalando cada parte:

C/C++

```
float calcularPromedio(float alturas[], int n) {
    float suma = 0;
    for (int i = 0; i < n; i++) suma += alturas[i];
    return suma / n;
}
```

Además existe el **PROTOTIPO** (o declaración) de la función, que es la firma de la función sin el cuerpo, terminada en punto y coma, y que se coloca antes de main() para que el compilador conozca la función antes de su definición completa.

3. DIFERENCIA ENTRE ARREGLO DE CARACTERES Y STRING

ARREGLO DE CARACTERES (char[]) - "cadena estilo C":

- Es un array de tipo char terminado obligatoriamente con el carácter nulo '\0'.
- Tiene tamaño fijo definido en tiempo de compilación (o reservado dinámicamente con new/malloc).
- No se puede redimensionar automáticamente.
- Las operaciones requieren funciones de la librería <cstring>: strcpy, strcat, strcmp, strlen, etc.
- Ejemplo: `char nombre[20] = "Anima";`

TIPO STRING (std::string, librería <string>):

- Es una clase de la STL que administra internamente un arreglo de caracteres de forma dinámica.
- Su tamaño crece o se reduce automáticamente según se necesite.
- Provee operadores sobrecargados (+, +=, ==) y métodos (length, find, substr, etc.).
- Ejemplo: `string nombre = "Anima";`

4. MÉTODOS DE LA CLASE STRING

Método	Funcionalidad	Ejemplo de uso
<code>max_size()</code>	Devuelve la cantidad máxima teórica de caracteres.	<code>cout << s.max_size();</code>
<code>compare()</code>	Compara lexicográficamente dos strings.	<code>a.compare(b);</code>
<code>empty()</code>	Indica si el string está vacío (longitud 0).	<code>if (s.empty()) ...</code>
<code>erase()</code>	Elimina caracteres a partir de una posición.	<code>s.erase(4, 6);</code>
<code>find()</code>	Busca la primera ocurrencia de una subcadena.	<code>s.find("mundo");</code>
<code>substr()</code>	Devuelve una subcadena (pos, lon).	<code>s.substr(5, 5);</code>

5. VECTOR EN C++: DEFINICIÓN Y CARACTERÍSTICAS

QUÉ ES: vector es un contenedor de la STL, definido en la librería `<vector>`, que representa un arreglo dinámico.

PRINCIPALES CARACTERÍSTICAS:

QUE ES: vector es un contenedor de la STL (Standard Template Library), definido en la librería `<vector>`, que representa un arreglo dinámico: almacena elementos del mismo tipo en posiciones de memoria contiguas, pero a diferencia de un array clásico, puede crecer o reducirse en tiempo de ejecución de forma automática.

PRINCIPALES CARACTERÍSTICAS:

- Tamaño dinámico: crece y se achica automáticamente (`push_back`, `pop_back`, `resize`, etc.), sin que el programador administre memoria manualmente.
- Almacenamiento contiguo: al igual que un array, permite acceso aleatorio a sus elementos en tiempo $O(1)$ mediante `[]` o `at()`.
- Es una plantilla (template): puede almacenar cualquier tipo de dato (`int`, `float`, `string`, objetos propios, etc.) usando `vector<T>`.
- Maneja el concepto de "size" (cantidad de elementos actuales) y "capacity" (memoria reservada internamente, que puede ser mayor al size para evitar reasignaciones constantes).

- Provee muchos metodos utiles ya incorporados (ver punto 9).

DECLARACION E INICIALIZACION:

```
vector<int> v1;           // vector vacio de int
vector<int> v2(5);        // 5 elementos en 0
vector<int> v3(5, 10);    // 5 elementos con valor 10
vector<int> v4 = {1, 2, 3, 4, 5}; // inicializado con lista
vector<int> v5(v4);       // copia de otro vector
vector<int> v6(v4.begin(), v4.end()); // por rango de iteradores
```

6. MANIPULACIÓN DEL TAMAÑO EN UN VECTOR

Existen diversos mecanismos para alterar la extensión de un vector:

- **RESIZE(N)**: ajusta la dimensión del contenedor a n elementos. Si n supera el tamaño previo, se insertan componentes adicionales (por defecto en 0); si es inferior, se descartan los elementos excedentes.
- **PUSH_BACK(VALOR)**: añade un nuevo dato al cierre del contenedor, incrementando su size. Puede derivar en una reasignación de memoria si se sobrepasa la capacidad actual.
- **POP_BACK()**: suprime el componente final, disminuyendo la cantidad de elementos en una unidad.
- **INSERT()**: permite la incorporación de uno o varios elementos en una ubicación específica, expandiendo el tamaño del vector.
- **ERASE()**: quita elementos del contenedor a partir de una posición indicada, reduciendo consecuentemente el size.
- **CLEAR()**: elimina la totalidad de los datos, dejando al vector con un tamaño de cero.
- **RESERVE(N)**: no altera el número de elementos (size), sino que asegura una capacidad (capacity) mínima para optimizar el rendimiento y prevenir realocaciones constantes al insertar datos conocidos.

7. ACCESO FUERA DE RANGO EN UN VECTOR

Las consecuencias de acceder a una posición inválida varían según el mecanismo empleado:

- **OPERADOR []:** no efectúa controles de límites. Su uso fuera de rango provoca un **COMPORTAMIENTO INDEFINIDO**, pudiendo retornar valores basura, corromper la memoria o finalizar abruptamente la ejecución sin advertencias del compilador.
- **MÉTODO AT():** valida los índices durante la ejecución. Ante un acceso erróneo, dispara una excepción `std::out_of_range`, permitiendo una gestión de errores estructurada mediante bloques try/catch.

En términos de buenas prácticas, se aconseja el uso de `at()` cuando la procedencia del índice es incierta (como entradas de usuario), reservando el operador `[]` únicamente para situaciones donde la validez del rango esté previamente garantizada, con el fin de optimizar el rendimiento.

8. LA CLASE VECTOR: QUE ES Y QUE FUNCIONALIDADES PROVEE

La clase vector es un contenedor secuencial de la STL, implementado como una plantilla generica (`template<class T> class vector`). Provee:

- Almacenamiento dinamico y contiguo de elementos de un mismo tipo.
- Gestion automatica de memoria (reserva, crecimiento y liberacion).
- Acceso aleatorio a elementos en tiempo constante (`operator[]`, `at`).
- Metodos para agregar/quitar elementos al final (`push_back`/`pop_back`) y en cualquier posicion (`insert`/`erase`).
- Metodos de consulta (`size`, `capacity`, `empty`, `front`, `back`).
- Iteradores (`begin()`, `end()`, `rbegin()`, `rend()`) que permiten recorrer el vector de forma generica, compatibles con los algoritmos de la STL (`sort`, `find`, `count`, etc. de `<algorithm>`).
- Sobrecarga de operadores de comparacion (`==`, `!=`, `<`, etc.) y de asignacion (`=`).
- Constructores multiples (vacío, por tamaño, por copia, por rango, por lista de inicializacion).

9. METODOS DE LA CLASE VECTOR

9. METODOS DE LA CLASE VECTOR		
Metodo	Funcionalidad	Ejemplo de uso
size()	Obtiene el tamaño (cantidad de elementos) actual	vector<int> v={1,2,3}; cout << v.size(); // 3
capacity()	Obtiene la capacidad reservada internamente	vector<int> v; v.reserve(10); cout << v.capacity(); // >=10
empty()	Comprueba si el vector esta vacio	vector<int> v; if (v.empty()) cout << "vacio";
push_back(valor)	Añade un elemento al final	vector<int> v; v.push_back(7); // v = {7}
pop_back()	Elimina el ultimo elemento	vector<int> v={1,2,3}; v.pop_back(); // v = {1,2}
front()	Devuelve una referencia al primer elemento	vector<int> v={5,6,7}; int x = v.front(); // 5
back()	Devuelve una referencia al ultimo elemento	vector<int> v={5,6,7}; int x = v.back(); // 7
at(posicion)	Devuelve una referencia al elemento en esa posicion, con chequeo de límites	vector<int> v={1,2,3}; int x = v.at(1); // 2
assign(inicio, fin)	Reemplaza el contenido con un rango de elementos	vector<int> a={1,2,3,4}, b; b.assign(a.begin(), a.begin()+2); // b={1,2}
erase(posicion)	Elimina el elemento en esa posicion	vector<int> v={1,2,3}; v.erase(v.begin()+1); // v = {1,3}
erase(inicio, fin)	Elimina un rango de elementos	vector<int> v={1,2,3,4,5}; v.erase(v.begin(), v.begin()+2); // v={3,4,5}
insert(posicion, valor)	Inserta un elemento en la posicion indicada	vector<int> v={1,2,4}; v.insert(v.begin()+2, 3); // v = {1,2,3,4}
insert(posicion, inicio, fin)	Inserta un rango de elementos en la posicion dada	vector<int> a={1,2}, b={9,9}; a.insert(a.end(), b.begin(), b.end());
clear()	Elimina todos los elementos del vector	vector<int> v={1,2,3}; v.clear(); // v.size() == 0
swap(otro_vector)	Intercambia el contenido con otro vector	vector<int> a={1,2}, b={9,9,9}; a.swap(b); // a={9,9,9}, b={1,2}

10. CLASE Y OBJETO EN C++

QUE ES UNA CLASE:

Una clase es una plantilla o molde definido por el programador que describe la estructura y el comportamiento que van a tener los objetos que se creen a partir de ella. Agrupa datos (atributos o miembros) y funciones que operan sobre esos datos (metodos), aplicando el principio de encapsulamiento propio de la Programacion Orientada a Objetos.

ESTRUCTURA DE UNA CLASE:

```

class NombreClase {
private:
    // Atributos (datos) - normalmente ocultos desde afuera
    tipo atributo1;
    tipo atributo2;

public:
    // Constructor(es): inicializan el objeto al crearlo
    NombreClase();

    // Metodos: funciones que definen el comportamiento
    tipo metodo1(parametros);
    tipo metodo2(parametros);

    // Destructor: se ejecuta al destruirse el objeto
    ~NombreClase();
};

```

Las secciones private, public (y protected en herencia) controlan el nivel de acceso a los miembros: los privados solo son accesibles desde dentro de la propia clase, los publicos son accesibles desde cualquier parte del programa que tenga un objeto de esa clase.

QUE ES UN OBJETO:

Un objeto es una instancia concreta de una clase: es la variable que se crea a partir del molde definido por la clase, y que ocupa memoria real con sus propios valores para cada atributo.

Ejemplo: NombreClase miObjeto; // miObjeto es un objeto de esa clase

PROPOSITO:

El proposito de usar clases y objetos es modelar entidades del mundo real (o del dominio del problema) de forma organizada, agrupando datos y comportamiento relacionado en una misma unidad. Esto permite:

- Encapsular la implementacion interna, exponiendo solo lo necesario.
- Reutilizar codigo (una clase se puede instanciar muchas veces).
- Facilitar el mantenimiento y la extension del programa mediante herencia y polimorfismo.
- Modelar el software de una forma mas cercana a como se piensa el problema real (por ejemplo, una clase Persona, Vehiculo, Cuenta Bancaria, etc.)